

Theory:

1. What is a class? What is an object?
2. Explain the difference between instance variables and class variables.
3. Define inheritance. What are its types in Python?
4. What is method overriding? Give an example.
5. What are static methods and class methods?

Coding:

1. Create a Student class with attributes name, roll_no, and marks, and a display() method.
2. Create Animal and Dog classes where Dog inherits from Animal and overrides the sound() method.
3. Create a BankAccount class with methods to deposit, withdraw, and check balance.

Q1ANS>A class is a user-defined data type in object-oriented programming (OOP). It acts as a template or blueprint that defines:

- **Attributes (data members):** These are variables that hold data.
- **Methods (member functions):** These are functions that define behavior.

An object is a real-world instance of a class. When you create an object, you allocate memory and give life to the structure defined by the class.

Q2ANS>

Quick Answer:

Instance variables are unique to each object of a class, while class variables are shared across all objects of that class.

Aspect	Instance Variable	Class Variable
Definition	Declared inside a class but outside methods, tied to a specific object	Declared inside a class with the static keyword (Java) or directly at class level (Python), shared across all objects
Scope	Exists separately for each object	Exists once per class, shared among all objects
Storage	Stored in the heap memory as part of the object	Stored in the method area (Java) or class namespace (Python)
Access	Accessed via the object reference (e.g., obj.var)	Accessed via the class name (e.g., Class.var) or object reference

| Value | Each object can have different values | All objects share the same value unless explicitly changed |

| Lifetime | Created when the object is instantiated and destroyed when the object is destroyed |
Created when the class is loaded and destroyed when the class is unloaded |

Q3ANS>**Inheritance** allows one class (child) to acquire properties and behaviors from another class (parent). It promotes code reuse and modular design.

Types of Inheritance in Python:

1. **Single Inheritance** – One child class inherits from one parent class.
 2. **Multiple Inheritance** – One child class inherits from multiple parent classes.
 3. **Multilevel Inheritance** – A class inherits from a child class which itself inherits from another class.
 4. **Hierarchical Inheritance** – Multiple child classes inherit from a single parent class.
 5. **Hybrid Inheritance** – A combination of multiple types of inheritance.
-

Q4ANS>Inheritance is a feature in object-oriented programming where a class (child) inherits attributes and methods from another class (parent). It promotes code reuse and logical hierarchy.

Types of Inheritance in Python:

1. Single Inheritance – One child class inherits from one parent class.
 2. Multiple Inheritance – A child class inherits from more than one parent class.
 3. Multilevel Inheritance – A class inherits from a child class which itself inherits from another class.
 4. Hierarchical Inheritance – Multiple child classes inherit from a single parent class.
 5. Hybrid Inheritance – A combination of multiple types of inheritance.
-

Q5Ans>Inheritance is a feature in object-oriented programming where a class (child) inherits attributes and methods from another class (parent). It promotes code reuse and logical hierarchy.

Types of Inheritance in Python:

1. Single Inheritance – One child class inherits from one parent class.
 2. Multiple Inheritance – A child class inherits from more than one parent class.
 3. Multilevel Inheritance – A class inherits from a child class which itself inherits from another class.
 4. Hierarchical Inheritance – Multiple child classes inherit from a single parent class.
 5. Hybrid Inheritance – A combination of multiple types of inheritance.
-

```
class Student:
    def __init__(self, name, roll_no, marks):
        self.name = name
        self.age = roll_no
        self.grade = marks

    def get_grade(self):
        print(f"Name:{self.name}")
        print(f"Roll No:{self.age}")
        print(f"Marks:{self.grade}")

s1 = Student("Saubhagya", 101, 92)
s1.get_grade()
```

```
Name:Saubhagya
Roll No:101
Marks:92
```

```
#Define the Animal class
class Animal:
    def sound(self):
        print("Animal makes a sound ")

#define the dog class inheriting from Animal
class Dog(Animal):
    def sound(self):
        print("Dog says: Woof woof!")
```